

Cómo se crea una historia Top-Down

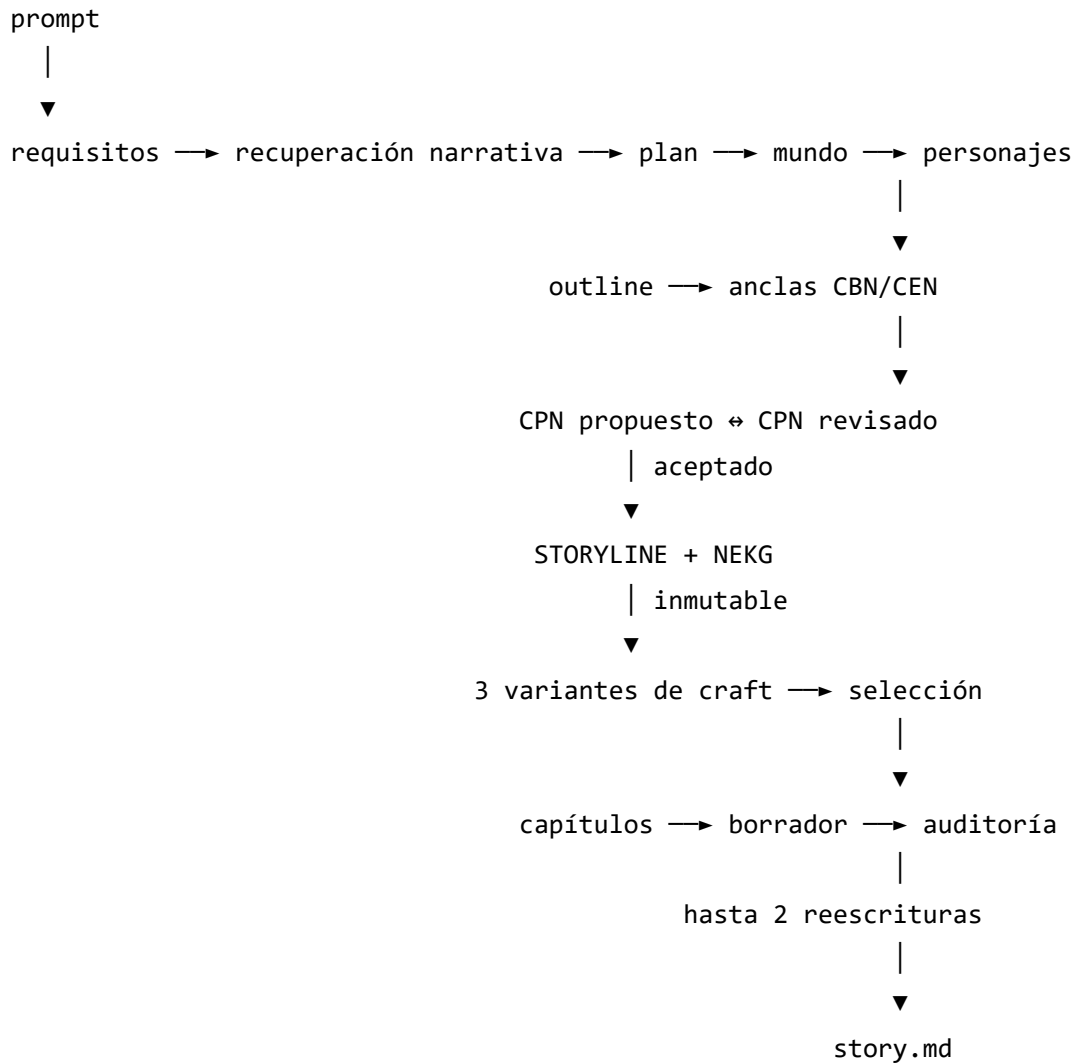
Esta guía describe la ruta de producción **Top-Down 3.0** tal como está implementada en `Models/Top-Down/src/asg_top_down` . Su propósito es explicar no solo qué ejecuta el sistema, sino también qué decisión se toma en cada etapa, por qué se toma y dónde observarla con el depurador.

La entrada pública es `storyGenerator` . No deben mezclarse con este recorrido clases o documentos de versiones anteriores: la implementación actual divide explícitamente la planificación de hechos y la planificación de craft.

1. La idea central

El sistema responde dos preguntas en momentos distintos:

1. **¿Qué ocurre?** STORYTELLER construye una secuencia causal de eventos aceptados.
2. **¿Cómo se cuenta?** El módulo de craft organiza promesas, progresos, pagos, evolución de personajes y ciclos de intento-fracaso alrededor de esos hechos.



La separación es deliberada. Si el craft pudiera aceptar, rechazar o modificar eventos, una solución elegante de estilo podría romper la causalidad. Si STORYTELLER decidiera también promesas y pagos, la evaluación de hechos y la de presentación se contaminarían. Por eso el craft se crea solamente cuando `storyline.json` ya existe.

También hay una segunda separación importante:

- El modelo decide significado: propone conflictos, eventos, revisiones y prosa.
- Python impone contratos: tipos, cantidades, IDs, orden, rangos, reintentos, persistencia y condiciones de aceptación.

El modelo aporta flexibilidad narrativa; las validaciones locales evitan que una respuesta convincente pero estructuralmente incorrecta avance silenciosamente.

2. Punto de entrada y configuración

La ejecución interactiva comienza en `asg_top_down.cli:main` :

```
.\.venv\Scripts\python.exe -m asg_top_down.cli
```

El CLI hace lo siguiente:

- 1. configura `stdout` y `stderr` como UTF-8;
- 2. solicita un `prompt` no vacío;
- 3. llama a `load_settings()` ;
- 4. construye `GeminiProvider` ;
- 5. construye `StoryGenerator` ;
- 6. llama a `generator.run(prompt)` y muestra el progreso;
- 7. imprime la ruta de `story.md` o un error público seguro.

La configuración se lee del `.env` de la raíz. Las variables relevantes son:

Variable	Decisión que controla	Valor predeterminado
GEMINI_API_KEY	credencial obligatoria	ninguno
GEMINI_MODEL	modelo de generación	gemini-3.5-flash-lite
GEMINI_EMBEDDING_MODEL	modelo de recuperación semántica	gemini-embedding-2
GEMINI_RPM_LIMIT	solicitudes por minuto disponibles	15
GEMINI_RPM_RESERVE	margen que no usa el proceso	1
GEMINI_TPM_LIMIT	límite de tokens por minuto; 0 lo desactiva	0
GEMINI_MAX_RETRIES	reintentos de transporte/cuota temporal	3
GEMINI_MAX_RETRY_DELAY	espera máxima entre reintentos	120 s
STORY_DEFAULT_WORDS	longitud si el usuario no da una	1500
STORY_MAX_CPN_RETRIES	reparaciones de cada candidato CPN	2

Variable	Decisión que controla	Valor predeterminado
STORY_MAX_ARTIFACT_RETRIES	reparaciones semánticas de artefactos	2

`find_project_root()` busca un ancestro que contenga `Models/` y `Stories/`. Esta decisión permite iniciar el programa desde la raíz o desde una carpeta interna sin cambiar las rutas de `.env`, caché y salida.

3. Recorrido completo, decisión por decisión

3.1. Interpretar el prompt

Si `StoryGenerator.generate()` recibe texto, `AnalystAgent.run()` lo convierte en un `StoryRequest` con:

- prompt original;
- título;
- idioma;
- género y tono;
- premisa;
- número objetivo de palabras;
- restricciones explícitas.

El idioma predeterminado es español y la extensión predeterminada procede de la configuración. Sin embargo, después de la respuesta del modelo se busca en el prompt una expresión como `1800 palabras` o `1800 words` y ese número se aplica de forma determinista.

Por qué: la longitud explícita del usuario no debe depender de que el modelo la extraiga correctamente. El esquema vuelve a validar que esté entre 300 y 20 000.

Después del análisis se crea el directorio de ejecución. No se crea antes porque el título normalizado forma parte de su nombre:

```
Stories/Top-Down/YYYYMMDD-HHMMSS-titulo-de-la-historia/
```

`request.json` es el primer artefacto y `metadata.json` comienza con estado `running`.

3.2. Recuperar conocimiento narrativo

`NarrativeSchemaRepository.retrieve()` consulta el catálogo SQLite de macrotramas, situaciones, arcos de personaje, beats, géneros y roles.

La consulta combina el prompt, la premisa, el género y el tono. Para cada entrada se calculan dos señales:

- coincidencia léxica mediante FTS5/BM25 y una comparación local de términos;
- similitud semántica mediante embeddings, cuando el proveedor está disponible.

La puntuación fusionada es:

$$0.4 \times \text{señal léxica} + 0.6 \times \text{señal semántica}$$

Se recuperan, por defecto, 2 macrotramas, 2 situaciones, 2 arcos, 8 beats, 2 géneros y 4 roles. Los resultados se ordenan de forma estable y se evita seleccionar variantes compatibles casi duplicadas cuando hay alternativas.

Por qué se recupera antes de planificar: reduce el espacio de búsqueda y ofrece vocabulario narrativo reutilizable sin obligar al planificador a copiar plantillas.

El catálogo orienta; no escribe la historia.

Por qué existe fallback léxico: una caída del servicio de embeddings no debe impedir toda la generación. Si embeddings falla, la selección continúa con FTS y coincidencia local.

Se guardan:

- `blueprint.json` : selección completa;
- `retrieval_trace.json` : consulta, puntuaciones, modelo de embedding y si realmente se usaron embeddings.

3.3. Diseñar el argumento causal

`PlannerAgent.run()` recibe `StoryRequest` y `NarrativeBlueprint` y crea `StoryPlanArtifact` :

- logline y tema;
- conflicto central;
- al menos tres pasos de progresión;

- final previsto;
- una macrotrama primaria y hasta dos secundarias.

`StoryGenerator._validate_plan()` comprueba que los IDs seleccionados existan en las macrotramas, situaciones o arcos recuperados.

Por qué se validan los IDs: el modelo puede inventar una etiqueta plausible que no pertenece al catálogo. Esa invención rompería la trazabilidad entre recuperación y planificación.

Si la validación falla, `_validated_artifact()` guarda el candidato y el error bajo `artifact_attempts/story_plan/`, adjunta feedback con el artefacto anterior y exige un reemplazo completo. Se permiten `STORY_MAX_ARTIFACT_RETRIES + 1` intentos totales.

3.4. Construir un mundo funcional

`WorldBuilderAgent.run()` produce escenario, época, reglas, lugares y atmósfera. Su instrucción exige que cada regla o lugar limite una decisión, cree una oportunidad o produzca una consecuencia.

Por qué: el worldbuilding decorativo aumenta contexto y costo sin ayudar a la causalidad. Un mundo funcional hace que las acciones posteriores tengan condiciones y consecuencias observables.

Esta etapa tiene validación de esquema Pydantic, pero no una validación semántica cruzada adicional mediante `_validated_artifact()`. Es importante distinguir la intención del prompt de una garantía determinista.

3.5. Diseñar personajes y su cambio observable

`CharacterDesignerAgent.run()` crea un reparto compacto. Cada personaje tiene un objetivo, motivación, conflicto, arco, importancia y arquetipo. Debe existir al menos un personaje principal.

Para cada principal se usan tres sliders:

- simpatía;
- competencia;
- proactividad.

El contrato `CharacterSliderArc` exige exactamente dos valores iniciales altos (7–10) y uno bajo (1–4). El bajo es el foco y debe terminar alto; los otros dos deben permanecer altos.

Por qué: un personaje competente, simpático y proactivo desde el principio tiene poco espacio de transformación; uno bajo en los tres puede resultar pasivo o difícil de seguir. Dos fortalezas sostienen el interés y una debilidad focal crea un arco legible. La dirección ascendente convierte el cambio en una condición comprobable.

Además del esquema, `validate_craft_characters()` comprueba que exista al menos un principal y que ninguno carezca de slider. Los candidatos inválidos entran en el mismo ciclo de guardar, explicar y regenerar.

3.6. Crear outline y presupuestos

`IncrementalPlotPlanner.outline()` crea antes que ningún nodo:

- una premisa;
- una sinopsis completa;
- capítulos ordenados;
- título y resumen de cada capítulo;
- fases de Freytag;
- presupuesto de palabras por capítulo.

`_validate_outline()` impone tres invariantes:

1. IDs de capítulo únicos;
2. órdenes consecutivos desde 1;
3. la suma de presupuestos coincide exactamente con `request.target_words`.

Por qué: la estructura global debe fijarse antes de generar eventos locales. De lo contrario, cada capítulo podría ser razonable por separado y aun así dejar sin espacio el clímax o exceder la longitud total.

Las fases de Freytag están restringidas por el tipo, pero el código actual no prueba que el conjunto global contenga todas las fases ni que aparezcan en un orden concreto.

3.7. Fijar el comienzo y final de cada capítulo

`IncrementalPlotPlanner.anchors()` genera exactamente dos eventos SVO por capítulo:

- **CBN, Chapter-Begin Node:** estado/evento observable de entrada;
- **CEN, Chapter-End Node:** resultado observable al que debe llegar el capítulo.

Se generan todos los pares antes de los nodos internos. `_validate_anchors()` exige una correspondencia uno a uno con los capítulos del outline.

Por qué se fija el final primero: un generador que solo conoce el pasado puede encadenar acciones interesantes que no llegan al destino del capítulo. Con un CEN predefinido, cada CPN se puede evaluar por su avance hacia un objetivo concreto.

La validación local garantiza cardinalidad e IDs. Los contenidos semánticos de CBN y CEN proceden del modelo y no atraviesan los siete controles usados para los CPN.

3.8. Construir la STORYLINE incremental

`IncrementalPlotPlanner.plan()` reinicia `StorylineState`, `NEKG` e historial. Luego procesa cada capítulo en orden.

Aceptar el CBN

El ancla inicial se convierte en un `PlotNode` con ID global `n_xxxx`, orden local y metadatos causales. En el primer capítulo no tiene arista entrante; en los siguientes se conecta el CEN anterior al nuevo CBN mediante `enables`.

El nodo se acepta inmediatamente, se aplica al `NEKG` y se guarda un checkpoint.

Calcular el máximo de CPN

Para un capítulo se permite como máximo:

```
max(1, min(10, ceil(chapter.target_words / 350)))
```

Por qué: el techo crece con el espacio disponible, evita pedir demasiados eventos para un capítulo corto y evita un bucle sin fin para uno largo. Es un límite de seguridad, no una cantidad obligatoria: el capítulo termina antes si un CPN aceptado ya conecta naturalmente con el CEN.

Proponer un CPN

Para cada slot, `_proposal()` pide un único evento SVO con:

- propósito y beat de referencia;

- precondiciones y efectos;
- intención del personaje;
- conflicto activo;
- cambios de estado opcionales.

El proponente conoce el capítulo, CBN, CEN, CPN aceptados en ese capítulo, conocimiento recuperado, número de slot y feedback del intento anterior. En el último slot recibe una instrucción explícita de crear el puente inmediato al CEN.

Por qué se propone uno a uno: el siguiente evento puede apoyarse en consecuencias ya aceptadas. Generar todos a la vez impediría incorporar rechazos y cambios de estado durante la planificación.

Revisar el CPN

`_review()` entrega al revisor:

- la propuesta;
- el capítulo y CEN objetivo;
- los 8 nodos aceptados más recientes de toda la historia;
- hasta 10 relaciones relevantes del NEKG.

El candidato final es `review.revised` cuando el revisor devuelve un reemplazo completo; en otro caso es la propuesta original. Para ser aceptado debe superar los siete controles:

1. soporte causal;
2. intención de personaje;
3. conflicto activo;
4. continuidad;
5. novedad;
6. avance hacia el final;
7. consistencia del mundo.

El validador de `PlotNodeReview` cambia `accepted` a falso si cualquiera de esos booleanos falla. En el último slot también es obligatorio `aligns_with_cen`.

Por qué el revisor puede reemplazar: algunos problemas se corrigen mejor reformulando el evento completo que devolviendo observaciones ambiguas. El sistema registra si se aceptó la propuesta o `review.revised`.

Aceptación y rechazo

Si se acepta:

1. el candidato se convierte en CPN;
2. se conecta desde el último nodo mediante una arista `causes` ;
3. se añade a `StorylineState` ;
4. se aplican evento y cambios de estado al NEKG;
5. se registra `AcceptedNodeRecord` ;
6. se guarda un checkpoint.

Si se rechaza, propuesta, review, candidato, procedencia e incidencias se añaden a `history.rejected` ; `STORYLINE` y `NEKG` permanecen intactos. El texto de los problemas se usa como feedback del siguiente intento.

Por qué un rechazo no toca el estado: permitir que un hecho descartado afecte el contexto contaminaría todas las decisiones posteriores y haría imposible auditar qué versión de la realidad narrativa se considera cierta.

Si se agotan `max_retries + 1` intentos de un slot, se lanza `StorylinePlanningError` . Una respuesta estructurada inválida también se registra como rechazo y consume un intento.

Cuando un CPN aceptado tiene `aligns_with_cen=True` , no se crean más slots para ese capítulo. Si se alcanza el techo sin alineación, la planificación falla.

Cerrar con el CEN

El CEN se materializa y se conecta con `causes` desde el último CPN. Después se reparte `chapter.target_words` entre CBN, CPN y CEN mediante división entera; el resto se añade al último nodo. El total del capítulo queda exacto.

La `STORYLINE` actual es lineal aunque se represente mediante nodos y aristas. Su `topological_order` es el orden de aceptación.

3.9. Mantener el NEKG

NEKG significa **Narrative Entity Knowledge Graph**. Es un estado local y auditable, no otro agente.

Al aplicar un nodo aceptado:

1. sujeto y objeto se normalizan a IDs sin acentos ni puntuación;
2. se crean las entidades que falten;
3. se añade la relación `sujeto --verbo--> objeto` con nodo y timestamp;
4. los cambios actualizan `location` , `possession` , `knowledge` , `situation` o `relationship` ;
5. `last_event_id` se actualiza para las entidades afectadas.

Para revisar una propuesta, `related(subject, object)` prioriza primero relaciones dirigidas exactamente de sujeto a objeto y luego relaciones incidentes recientes, hasta un máximo de diez.

Por qué: el revisor no necesita recibir toda la historia para comprobar, por ejemplo, quién posee un objeto o qué sabe un personaje. El subconjunto local reduce ruido y costo. El NEKG aporta hechos, pero no ejecuta inferencias ni demuestra por sí solo la coherencia; esa decisión sigue siendo semántica.

Limitaciones actuales:

- el último valor de un atributo reemplaza al anterior;
- no hay resolución de correferencias (Ana , ella , la capitana);
- CBN y CEN no aportan cambios de estado explícitos;
- el grafo no se vuelve a extraer de la prosa final.

3.10. Crear tres variantes de craft

Solo después de cerrar STORYLINE, `CraftVariantPlannerAgent.run()` genera exactamente `variant-1` , `variant-2` y `variant-3` . Cada una debe tener una estrategia distinta y:

- una línea PPP maestra: `promise`, `progress`, `payoff`;
- de cero a dos subtramas PPP completas;
- una línea PPP local por capítulo;
- hitos `start` , `transition` y `end` para el slider focal de cada principal;
- una cantidad exacta de ciclos Yes-but/No-and con consecuencias persistentes.

La cantidad de ciclos es:

```
max(2, min(7, ceil(target_words / 2000)))
```

La promesa maestra debe comenzar en el primer capítulo y su pago debe ocurrir en el último. Todos los puntos deben estar ordenados, cada capítulo debe avanzar líneas

globales conocidas y cada personaje principal debe tener exactamente sus tres hitos.

El craft no puede contener IDs `n_XXXX` ni las palabras CBN, CPN o CEN.

Por qué se prohíben referencias a nodos: el craft debe describir efectos para el lector y comportamientos observables, no acoplarse a detalles internos de la STORYLINE. Así una variante puede redactarse después sin replanificar hechos.

Por qué se crean tres variantes: la misma cadena causal puede sostener distintas estrategias de expectativa, énfasis y arco. Compararlas antes de redactar es más barato que escribir tres historias completas desde el inicio.

Cada variante válida se guarda de inmediato en:

```
craft/variants/variant-N/plan.json
craft/variants/variant-N/global.json
craft/variants/variant-N/chapters/<chapter-id>.json
```

3.11. Seleccionar la variante canónica

`CraftVariantSelectorAgent.run()` elige una variante según:

- fidelidad a las restricciones del usuario;
- ajuste causal a la STORYLINE;
- progresión global y por capítulo;
- pagos preparados;
- cambio observable del slider focal.

Python comprueba que el ID elegido exista. La selección y su justificación se guardan en `craft/selection.json`.

Por qué el selector no puntúa: una puntuación única escondería compensaciones entre criterios. La decisión conserva una justificación textual y las alternativas no se destruyen.

3.12. Redactar capítulos

`_render_to_prefix()` recorre los capítulos del outline. Antes de escribir cada uno, reconstruye un NEKG de escritura aplicando los nodos y cambios aceptados hasta ese capítulo.

`ChapterWriterAgent.run()` recibe:

- solicitud, plan, mundo y personajes;
- craft global de la variante y craft local del capítulo;
- hitos de personaje y ciclos try-fail correspondientes;
- nodos del capítulo y aristas que los tocan;
- estado NEKG acumulado;
- el capítulo anterior completo, o `none` para el primero;
- presupuesto aproximado del capítulo.

Por qué solo se incluye el capítulo anterior completo: ofrece continuidad de voz y detalles inmediatos sin hacer crecer el contexto con todo el manuscrito.

El escritor devuelve solo el cuerpo. `_canonical_chapter()` elimina un encabezado que el modelo haya añadido y coloca exactamente `## <título>`. Cada capítulo se guarda al terminar y todos se unen en `draft.md`.

3.13. Auditar y reescribir

`CraftCriticAgent` no asigna una nota general. `audit_questions()` construye preguntas con IDs estables para verificar:

- promesa, progreso y pago de cada línea PPP global;
- promesa, progreso y pago de cada capítulo;
- comienzo, transición, decisión y final del arco focal de cada principal;
- resultado y persistencia de cada ciclo try-fail;
- cada restricción del usuario;
- preservación de causalidad;
- ausencia de terminología de planificación.

La mayoría son bloqueantes. Son consultivas `earned` de cada pago global y la ausencia de andamiaje. Si el crítico omite una pregunta, `normalize_audit()` la convierte en un fallo explícito; no se interpreta el silencio como aprobación.

También se comprueba la longitud total con tolerancia:

```
mínimo = ceil(objetivo × 0.90)
máximo = floor(objetivo × 1.20)
```

Si hay fallos bloqueantes o la longitud está fuera del rango, el reescritor recibe la historia completa, las instrucciones de todos los fallos y, cuando corresponde, una instrucción determinista de longitud. Puede haber hasta dos reescrituras, por lo que se auditan como máximo tres versiones: intento 0, 1 y 2.

Por qué se reescribe la historia completa: un pago o una consecuencia modificada al final suele requerir preparación en capítulos anteriores. Un parche local podría resolver la frase señalada y romper continuidad o causalidad en otro lugar.

La versión entregada se elige en este orden:

- 1. menor cantidad de fallos bloqueantes;
- 2. menor distancia al rango de longitud;
- 3. menor cantidad de fallos consultivos;
- 4. en empate, versión más reciente.

Por qué se conserva la mejor versión: si una auditoría o reescritura tardía falla, ya existe ficción utilizable. El sistema emite `quality_warning.json` en vez de perder todo el trabajo.

Una distinción importante: `draft.md` y `chapters/*.md` conservan la primera redacción por capítulos, mientras `story.md` puede ser una reescritura completa posterior. La auditoría total de longitud corresponde a `story.md` ; las entradas de longitud por capítulo se calculan sobre los capítulos iniciales.

4. Reintentos: cuatro capas diferentes

No todos los reintentos significan lo mismo:

Capa	Qué repara	Límite normal
<code>GeminiProvider._generate()</code>	red, 408, 429 y 5xx temporales	<code>GEMINI_MAX_RETRIES</code>
<code>generate_structured()</code>	JSON que no satisface el esquema Pydantic	1 reparación adicional
<code>_validated_artifact()</code>	contrato semántico cruzado de plan, personajes, outline,	<code>STORY_MAX_ARTIFACT_RETRIES + 1</code> intentos totales

Capa	Qué repara	Límite normal
	anclas, variantes y selección	
bucle de CPN	causalidad y siete controles semánticos de un evento	STORY_MAX_CPN_RETRIES + 1 intentos por slot

Separarlas permite diagnosticar si el problema fue transporte, forma JSON, coherencia de un artefacto completo o calidad de un evento concreto.

Todas las llamadas registran duración, tokens, esperas y reintentos en `llm_usage.json` . La API key y los prompts no se copian a reportes públicos.

5. Persistencia y artefactos

Una ejecución completa tiene esta forma:

```

run/
├─ metadata.json
├─ request.json
├─ blueprint.json
├─ retrieval_trace.json
├─ story_plan.json
├─ world.json
├─ characters.json
├─ outline.json
├─ chapter_anchors.json
├─ planning_checkpoint/
│   ├─ storyline.json
│   ├─ nekg.json
│   └─ node_reviews.json
├─ storyline.json
├─ nekg.json
├─ node_reviews.json
├─ craft/
│   ├─ selection.json
│   ├─ variants.json
│   └─ variants/
│       ├─ variant-1/
│       │   ├─ plan.json
│       │   └─ global.json
│       └─ chapters/<chapter-id>.json
│           └─ archivos de redacción si fue renderizada
├─ variant-2/
└─ variant-3/
├─ chapters/chapter-XXX.md
├─ draft.md
├─ craft_revisions/
├─ craft_audit.json
├─ diagnostic_audit.json
├─ craft_revision_history.json
├─ length_audit.json
├─ llm_usage.json
├─ quality_warning.json          # solo si hubo advertencias
├─ error_report.json            # solo si la ejecución falló
└─ story.md

```

La variante seleccionada contiene bajo su propia carpeta `chapters/chapter-XXX.md` , `draft.md` , `craft_revisions/` , auditorías, historial, uso y `story.md` . Esos resultados

se reflejan además en la raíz para que CLI, consola y Telegram encuentren una salida canónica estable.

Los checkpoints se sobrescriben después de cada CBN, aceptación, rechazo y CEN; por eso contienen el estado parcial más reciente. El historial de revisiones conserva el detalle de intentos anteriores.

6. Fallos, `resume()` y variantes alternativas

Si una etapa temprana falla, `ArtifactRepository.fail()` cambia `metadata.json` a `failed` y escribe un `error_report.json` seguro. Los artefactos previos no se borran.

`resume(run_id)` tiene una semántica concreta:

- si ya existe `story.md`, devuelve esa ejecución sin hacer llamadas;
- si no existe, lee `request.json` y comienza una ejecución nueva en otro directorio.

No continúa todavía desde `planning_checkpoint/` dentro del mismo directorio. El nombre `resume` significa “recuperar el pedido y volver a ejecutar”, no “reanudar en la instrucción exacta donde falló”.

`render_variant(run_id, "variant-2")` sí reutiliza la planificación de un run 3.0:

1. valida que existan los artefactos obligatorios;
2. valida otra vez el plan de craft elegido;
3. redacta y audita esa variante sin llamar a análisis, recuperación, mundo, personajes, outline, anclas, CPN ni selector;
4. devuelve la carpeta de la variante y no cambia `craft/selection.json` ni `story.md` de la raíz.

Si la variante ya tiene `story.md`, la operación es idempotente y retorna de inmediato.

7. Configuración de depuración en VS Code

El archivo `.vscode/launch.json` contiene un único recorrido llamado **Crear historia Top-Down paso a paso (Gemini)**. Es la ruta real e interactiva: carga `.env`, utiliza el intérprete del entorno virtual, ejecuta `asg_top_down.cli`, llama a Gemini y guarda la historia bajo `Stories/Top-Down/`.

`stopOnEntry` está activado para que el depurador se detenga antes de ejecutar el CLI. Desde ahí se puede recorrer el programa con `F10` y entrar en las funciones propias del proyecto con `F11`. `justMyCode` permanece activo para no entrar accidentalmente en miles de líneas internas de `debugpy`, `Pydantic` o el SDK de Google; las llamadas a Gemini sí se observan dentro de `GeminiProvider`.

En VS Code:

- 1. abre “Run and Debug” con `Ctrl+Shift+D` ;
- 2. selecciona **Crear historia Top-Down paso a paso (Gemini)**;
- 3. coloca los breakpoints indicados en la sección siguiente;
- 4. pulsa `F5` ;
- 5. usa `F10` para avanzar sin entrar y `F11` para entrar en una llamada.

La consola integrada es obligatoria porque el CLI solicita el prompt con `input()`. Cuando la ejecución llegue a esa línea, escribe allí la descripción de la historia y presiona `Enter`.

8. Breakpoints principales

No hace falta detenerse en cada setter de `Pydantic`. Este recorrido muestra las decisiones que cambian el estado narrativo.

Orden	Archivo y función	Qué observar
1	<code>cli.py</code> → <code>main()</code>	prompt, settings , proveedor y llamada pública <code>generator.run()</code>
2	<code>generator.py</code> → <code>StoryGenerator.generate()</code>	creación del run y orden completo de etapas
3	<code>generator.py</code> → <code>_validated_artifact()</code>	candidate , validate , feedback , issues y reintentos semánticos
4	<code>narrative_db.py</code> → <code>retrieve()</code>	query , fts_scores , vectors , ranked y selections
5	<code>incremental.py</code> → <code>IncrementalPlotPlanner.plan()</code>	chapter , slot , attempt , proposal , review , candidate y aligned

Orden	Archivo y función	Qué observar
6	incremental.py → StorylineState.accept()	invariantes antes de añadir un nodo y sus aristas
7	nekg.py → NarrativeEntityGraph.apply()	cómo solo un nodo aceptado cambia entidades, relaciones y estado
8	generator.py → _save_variant_plan() y _validate_selection()	las tres alternativas y la elección canónica
9	generator.py → _render_to_prefix()	reconstrucción del NEKG y contexto enviado por capítulo
10	agents/writer.py → ChapterWriterAgent.run()	filtros de nodos, aristas, craft, hitos y capítulo anterior
11	generator.py → _review_draft()	auditoría, condición de reescritura y selección de la mejor versión
12	storage.py → save_data() o save_text()	qué artefacto se persiste y en qué momento

Para estudiar el núcleo incremental con menos interrupciones, coloca dos breakpoints condicionales dentro de `IncrementalPlotPlanner.plan()` :

- en el bloque de aceptación: `review.accepted and not final_without_alignment ;`
- en el bloque de rechazo: `not review.accepted or final_without_alignment .`

Expresiones útiles en “Watch”:

```
repository.run_dir
request.target_words
[c.id for c in outline.chapters]
(chapter.id, slot, attempt, maximum)
candidate.model_dump()
review.model_dump()
[n.id for n in self.state.nodes]
self.nekg.artifact().model_dump()
selection.selected_variant_id
audit.failed_blocking_ids
[(i, len(a.failed_blocking_ids), ok) for i, _, a, ok in versions]
```

Entrar con `F11` en `GeminiProvider._generate()` permite distinguir espera de cuota, llamada remota y validación. Cada continuación puede efectuar una llamada real y consumir cuota; conviene inspeccionar primero `system_instruction`, `prompt` y `operation`, y después continuar.

9. Recorrido de depuración recomendado

Primera pasada: arquitectura

Usa la configuración de Gemini y detente solo en:

1. `StoryGenerator.generate()` ;
2. `IncrementalPlotPlanner.plan()` ;
3. `_render_to_prefix()` ;
4. `_review_draft()` .

Avanza con `F10` . El objetivo es ver cuándo nacen `request`, `storyline`, `variants`, `draft` y `story` .

Segunda pasada: aceptación causal

Reinicia la generación y usa los breakpoints condicionales de aceptación/rechazo. Compara:

- `proposal` frente a `review.revised` ;
- `history.rejected` antes y después;
- `self.state.nodes` y `self.nekg.artifact()` .

El punto esencial es comprobar que un rechazo aumenta el historial, pero no el grafo; una aceptación actualiza ambos.

Tercera pasada: craft y calidad

Detente en `_save_variant_plan()`, `ChapterWriterAgent.run()` y `_review_draft()` . Comprueba que:

- las tres variantes comparten `STORYLINE` pero no estrategia;
- el escritor recibe solamente el craft seleccionado;
- una primera auditoría con fallo genera una reescritura;
- `selected_attempt` identifica la versión final, que no tiene que ser la última.

Cuarta pasada: llamadas al proveedor

Introduce un prompt corto y revisa:

- `AnalystAgent.run()` para requisitos;
- `GeminiProvider.generate_structured()` para contratos JSON;
- `_validated_artifact()` para reparaciones semánticas;
- `llm_usage.json` para costo, espera y reintentos.

10. Pruebas rápidas independientes del depurador

La configuración de VS Code genera una historia real con Gemini. Si se desea validar el flujo sin consumir API, esta prueba independiente usa un proveedor falso:

```
.\.venv\Scripts\python.exe -m pytest -q -s `
Models\Top-Down\tests\test_top_down_v3.py::test_generator_builds_variants_rewrites_blocking_fi
```

Todas las pruebas del paquete:

```
.\.venv\Scripts\python.exe -m pytest -q Models\Top-Down\tests
```

Estas pruebas usan proveedores falsos donde corresponde; no necesitan la API para validar el flujo principal.

11. Índice del código

- [CLI](#)
- [Orquestador de producción](#)
- [Planificación incremental](#)
- [Recuperación narrativa](#)
- [Grafo NEKG](#)
- [Reglas de craft](#)
- [Agentes](#)
- [Esquemas y validadores Pydantic](#)
- [Proveedor Gemini](#)
- [Persistencia](#)
- [Configuración](#)
- [Prueba integral v3](#)

12. Resumen en pseudocódigo

```
request = analyze(prompt)
repo = create_run(request.title)

blueprint = retrieve_catalog(request)          # embeddings + léxico; fallback local
plan = validate_or_regenerate(plan_story(request, blueprint))
world = build_functional_world(request, plan)
characters = validate_or_regenerate(design_characters(request, plan, world))
outline = validate_or_regenerate(create_outline(request, plan, blueprint))
anchors = validate_or_regenerate(create_all_CBN_and_CEN(outline, world, characters))

storyline = empty_storyline()
nekg = empty_graph()

for chapter in outline:
    accept(CBN)
    nekg.apply(CBN)
    checkpoint()

    for slot in adaptive_ceiling(chapter):
        for attempt in allowed_attempts:
            proposal = propose_one_CPN(accepted_history, CEN_target)
            review = review_seven_checks(proposal, recent_storyline, related_nekg)
            candidate = review.revised or proposal

            if review.accepted and (not_final_slot or review.aligns_with_cen):
                accept(candidate)
                nekg.apply(candidate, candidate.state_changes)
                checkpoint()
                break

            record_rejection_without_changing_story_state()
            checkpoint()
        else:
            fail_planning()

    if review.aligns_with_cen:
        break

require_alignment_with_CEN()
accept(CEN)
```

```
nekg.apply(CEN)
allocate_chapter_words_exactly()
checkpoint()
```

```
variants = validate_or_regenerate(create_three_craft_variants(frozen_storyline))
selection = validate_or_regenerate(select_one_variant(variants))
draft = write_chapters(selection, storyline, nekg, previous_chapter_only=True)
versions = audit_and_rewrite_up_to_two_times(draft)
story = choose_best(versions, blocking_failures_then_length_then_advisories)
persist_and_complete(story)
```

La regla que permite entender todo el proyecto es: **ningún hecho se vuelve verdad narrativa hasta que el revisor lo acepta; ningún recurso de craft puede cambiar esa verdad después.**